编译原理研讨课实验报告

实验二 PR001：CACT 词法与语法分析

组号：1
成员及分工：韩旭成 (组长)，张高翔

1 任务说明

本次实验的目标是根据 CACT 语言规范编写 ANTLR 文法，生成 C++ 词法和语法分析器，并将它们组织成一个简单的编译器前端。程序从命令行接收源文件路径：输入没有词法、语法错误时返回 0，否则返回非零值。本阶段主要完成词法与语法检查，语义分析、中间表示和代码生成将在后续实验中实现。

2 分工

- 韩旭成（组长）：负责阅读 CACT 语言规范，设计词法规则和语法规则，完成声明、函数、控制语句及表达式优先级等文法的实现。
- 张高翔：负责程序入口和错误处理、测试用例运行与结果整理，配置本地及集群实验环境，并整理实验报告与项目文档。

3 实验设计

3.1 设计思路

我们先根据语言规范编写 `CACT.g4`，再使用 ANTLR 生成 C++ 词法、语法分析器。程序读取源文件后，由词法分析器产生 token 流，语法分析器再根据文法检查这些 token。分析过程中如果发现错误，就通过程序退出状态向测试脚本报告。

构建时，CMake 会调用课程提供的 ANTLR 4.13.1 生成器，并将生成的代码与配套 C++ runtime 编译链接。生成文件放在构建目录中，不需要手工修改；文法发生变化后，构建过程会重新生成分析器。

3.2 词法分析

编写词法规则时，我们把关键字规则放在标识符规则之前。这样，当两条规则匹配到相同长度的文本时，`if` 等关键字会被优先识别为关键字，而不是普通标识符。标识符以 ASCII 字母或下划线开头，后续可包含数字。整数支持十进制、八进制和十六进制；非零十进制整数不允许前导零。浮点数支持小数点形式和指数形式及可选的 `f/F` 后缀；单纯整数加 `f` 不构成浮点数。数值前的正负号由文法处理，指数内部的符号属于浮点词法单元。

字符常量包含一个 ASCII 字符或支持的转义序列，程序会拒绝空字符、多字符、未知转义和非 ASCII 字符。注释在词法分析阶段跳过：单行注释遇到换行或文件结尾时结束，多行注释在第一个结束符处结束，不支持嵌套。

3.3 语法设计与表达式优先级

我们将编译单元写成一个或多个声明或函数定义，并在末尾匹配文件结束符，确保整个输入文件都参与检查。声明初值采用常量字面量与递归初始化列表；数组声明维度采用整数字面量。函数形参最外层数组维度可以省略，内层维度需要给出。

为了在文法中体现运算符优先级，我们将算术表达式依次分为 `primaryExp`、`unaryExp`、`mulExp` 和 `addExp`。乘除取余高于加减，同级循环体现左结合，一元规则递归体现右结合。条件表达式进一步划分关系、相等、逻辑与、逻辑或层级；逻辑非处于一元运算层级。赋值单独建模为语句，不支持连续赋值或逗号表达式。普通表达式与条件表达式分开，`else` 就近匹配未匹配的 `if`。

3.4 错误处理

主程序先检查参数个数及文件是否可打开。词法分析器和语法分析器共用错误监听器，记录错误并输出位置及诊断信息。调用 `tokens.fill()` 扫描完整文件，以发现尾部非法字符，再解析匹配 EOF 的编译单元。

情况	退出状态
词法、语法均正确	0
词法或语法错误	1
参数、文件或其他分析异常	2

ANTLR 遇到语法错误后可能尝试恢复并继续分析。如果只看解析函数是否正常返回，就可能把错误输入误判为正确。因此，程序在结束前还会检查监听器状态和解析器的错误计数。

3.5 规范取舍

课程规范中初始化的文法与示例存在不完全一致的地方。实现时，我们结合文字说明和公开样例，将初值限制为字面量及初始化列表，拒绝使用变量、函数调用或算术表达式作为初值。`return` 语句要求分号；十进制整数支持多位。

课程规范的关键字表中列出了 `double`，但基本类型说明只列出三种类型。根据小组要求，本实现将 `double` 加入基本类型规则，使其可以用于变量、数组、函数返回值和形参。这属于文法层面的扩展，最终应以课程后续要求为准；类型匹配仍留待后续语义分析。

3.6 关键代码与分析

下面选取文法文件和主程序中几个与实验要求直接相关的部分，说明规则设计及错误处理方式。代码片段来自当前实现。

数值常量的词法规则。整数规则分别处理十六进制、八进制和十进制；浮点数规则区分小数点形式和指数形式。例如，077 按八进制规则匹配，3E2f 按浮点数规则匹配，而单独的 3f 不满足浮点规则：

```
FLOAT_CONST
    : (DIGIT+ '.' DIGIT* | '.' DIGIT+) EXPONENT? [fF]?
    | DIGIT+ EXPONENT [fF]?
    ;
INT_CONST: '0' [xX] HEX_DIGIT+ | '0' [0-7]* | [1-9] DIGIT*;
fragment EXPONENT: [eE] [+-]? DIGIT+;
```

这里的正负号不放在整数和浮点数的主体规则中，而由一元表达式处理；指数中的正负号则保留在 EXPONENT 中。

基本类型和声明规则。 bType 被变量声明、常量声明、函数返回类型和形参复用，因此在这一处加入 DOUBLE 后，这些位置都能识别 double：

```
bType: INT | FLOAT | DOUBLE | CHAR;
varDecl: bType varDef (COMMA varDef)* SEMI;
funcDef: funcType IDENT LPAREN funcFParams? RPAREN block;
funcType: VOID | bType;
```

表达式优先级。在以下规则中，addExp 由一个或多个 mulExp 组成，因此乘、除、取余先于加减结合。同一层级通过循环处理连续运算：

```
mulExp: unaryExp ((MUL | DIV | MOD) unaryExp)*;
addExp: mulExp ((PLUS | MINUS) mulExp)*;
```

例如，对于 1 + 2 * 3，语法分析时 2 * 3 在一个 mulExp 内匹配，再与左侧的 1 组成 addExp，从而体现乘法的较高优先级。

文件输入。主程序要求恰好提供一个源文件路径，并在读取前检查文件是否打开。参数或文件错误直接返回非零状态，避免将无法读取的文件当作合法输入：

```
if (argc != 2) {
    std::cerr << "Usage: " << argv[0] << " <source.cact>\n";
    return 2;
}
std::ifstream source(argv[1], std::ios::binary);
if (!source) {
    std::cerr << "Cannot open input: " << argv[1] << '\n';
    return 2;
}
```

错误监听器。词法和语法分析使用同一个监听器。每次收到错误时，它会设置标志并输出位置；这样即使 ANTLR 恢复后继续分析，主程序也能根据标志给出非零退出状态：

```
class ErrorListener final : public antlr4::BaseErrorListener {
public:
    bool failed = false;
```

```
    void syntaxError(antlr4::Recognizer *, antlr4::Token *, size_t line,
                     size_t column, const std::string &message,
                     std::exception_ptr) override {
        failed = true;
        std::cerr << line << ':' << column + 1 << ": " << message << '\n';
    }
};
```

完整输入与错误返回。编译单元要求匹配 EOF，主程序先填充 token 流，检查词法错误后再进行语法分析。这样既能检测合法程序后面的非法字符，也能在解析结束后根据错误记录返回正确的状态：

```
compUnit: (decl | funcDef)+ EOF;
```

```
tokens.fill();
if (errors.failed) return 1;
CACTParser parser(&tokens);
parser.removeErrorListeners();
parser.addErrorListener(&errors);
parser.compUnit();
return errors.failed || parser.getNumberOfSyntaxErrors() != 0 ? 1 : 0;
```

4 实验实现与结果

4.1 实验环境

本地环境为 Ubuntu 24.04 WSL、G++ 13.3.0、Java 17 和 CMake 3.28.3；ANTLR 生成器与 C++ runtime 均为 4.13.1。课程集群使用 G++ 11.4.0、Java 17 和 CMake 3.22.1。

4.2 测试结果

测试类别	通过数
公开词法语法样例	27/27
独立边界用例	64/64
命令行用例	3/3
合计	94/94

我们在本地使用课程原有测试脚本运行了 27 个公开样例，没有出现误判。补充测试覆盖了数值、字符、注释、表达式，以及 `double` 变量、数组、函数和错误形参等情况。上述测试均已通过，逐项记录保存在项目中。

4.3 复现方法

```
cmake -S . -B build
cmake --build build -j 4
cd test
bash test_syntax.sh
```

课程脚本逐一调用编译器；若实际退出状态与样例名称中的预期不符，输出 `**Error!`。也可在仓库根目录执行 `./build/compiler input.cact`，随后用 `echo $?` 查看退出状态。

5 总结

本次实验完成了 CACT 词法、语法分析器的编写、构建和测试。实现过程中，表达式优先级通过分层文法解决；错误处理则需要同时考虑词法错误、语法错误以及 ANTLR 的错误恢复。当前版本在本地通过了公开样例和补充用例，更新后的集群测试仍有待验证。后续实验可以在此基础上使用 Visitor 接口，实现符号表、类型和控制流检查。